

Guía de Estudio — Tema 07: Paradigma Lógico — Clase 2+3

Para el alumno: este documento es tu compañero de estudio autónomo. Cubre en profundidad todo lo visto en la clase doble y te prepara para el TP y el parcial.

Materia: Paradigmas y Lenguajes de Programación 2026 **Docente:** Matías Gel — UNTDF / IDEI **Duración de la clase:** 240 min | **Ciclo:** 2 de 2 del módulo III **Fecha:** 2026-04-21

0. Cómo usar esta guía

Esta guía NO reemplaza la clase — la **expande**. Su estructura:

1. **Objetivos** (qué vas a saber al terminar)
2. **Conceptos previos** (qué tenés que saber antes)
3. **Desarrollo teórico** (el 80% del documento)
4. **Ejemplos trabajados paso a paso**
5. **Puntos clave** (lo que tenés que recordar sí o sí)
6. **Autoevaluación** (20 preguntas con respuesta al final)
7. **Glosario**
8. **Referencias**

Tiempo estimado de lectura activa: 4 horas distribuidas en 2–3 sesiones de 90 min.

Método recomendado: - Leé 1 sección. - Abrí SWISH (<https://swish.swi-prolog.org/>) y **tipeá cada ejemplo**. - Modificá los ejemplos (cambiale algo) y observá qué cambia. - Cuando termines una sección, pasá a la siguiente **sólo si podés explicarla en voz alta**.

1. Objetivos de Aprendizaje

Al finalizar el estudio de este tema deberías poder:

#	Objetivo	Nivel Bloom
1	Definir unificación y aplicar su algoritmo a mano	Recordar, Aplicar
2	Trazar árboles SLD de consultas con 2+ niveles de recursión	Analizar
3	Distinguir entre <code>=</code> , <code>==</code> , <code>==:</code> , <code>is/2</code>	Comprender
4	Explicar cómo funciona el backtracking y el trail	Comprender
5	Decidir cuándo usar <code>!</code> (corte verde vs. rojo)	Evaluar
6	Justificar el uso de <code>dif/2</code> en lugar de <code>\+</code> con variables	Evaluar
7	Escribir <code>append/3</code> , <code>member/2</code> , <code>reverse/2</code> sin mirar apuntes	Aplicar
8	Convertir una recursión ingenua en una con acumulador	Crear
9	Elegir entre <code>findall</code> , <code>bagof</code> y <code>setof</code> en cada caso	Evaluar
10	Modelar un dominio de conocimiento y consultarlo	Crear
11	Resolver un puzzle de restricciones (generate-and-test o CLP(FD))	Crear
12	Situar Prolog en el ecosistema 2026 (Datalog, neuro-simbólico)	Comprender

2. Conocimientos Previos

Antes de abordar este tema deberías dominar:

- **De Tema 06 (Clase 1):** hechos, reglas, consultas, cláusulas de Horn, términos (átomo, número, variable, estructura), recursión `ancestro/2`.
- **De paradigma funcional:** recursión con patrón base/recursivo, listas como `[H|T]`.
- **Lógica proposicional y de predicados:** $\forall$, $\exists$, implicación, conjunción. Nivel informal.
- **Complejidad algorítmica:** qué significa $O(n)$, $O(n^2)$, $O(2^n)$.

Si alguno de estos temas está flojo, revisá: - Sebesta, *Concepts of Programming Languages*, cap. 15 (paradigma funcional) y cap. 16.1–16.2 (lógica). - Tema 06 del curso.

3. Desarrollo Teórico

3.1 Unificación: el corazón de Prolog

La **unificación** es el único mecanismo de intercambio de información en Prolog. No hay asignación, no hay retorno de función. Todo ocurre a través de unificación.

Definición formal: unificar dos términos t_1 y t_2 es encontrar una sustitución θ (función variables $\rightarrow$ términos) tal que $t_1\theta = t_2\theta$ sintácticamente.

Tipos de términos: - **Constantes** (átomos, números): `ana`, `5`, `'Buenos Aires'` - **Variables:** `X`, `Persona`, `_` - **Estructuras** (funtor + argumentos): `padre(juan, maria)`, `punto(3, 4)`, `.(cabeza, cola)` — las listas son azúcar de esto

3.1.1 Los cuatro casos

1. **Átomos iguales:** `ana = ana` $\rightarrow$ éxito con $\theta = \{\}$.
2. **Variable libre + término:** `X = ana` $\rightarrow$ éxito con $\theta = \{X/ana\}$.
3. **Dos variables:** `X = Y` $\rightarrow$ éxito con $\theta = \{X/Y\}$ (ambas quedan ligadas).
4. **Dos estructuras con mismo funtor y aridad:** `f(a1, ..., an) = f(b1, ..., bn)` $\rightarrow$ unificar recursivamente cada par (a_i, b_i) .

Si ninguno aplica $\rightarrow$ **falla**.

3.1.2 El algoritmo de Robinson (1965)

```

unify(t1, t2):
    si t1 == t2:                                # idénticos
        return  $\theta$  actual
    si t1 es variable libre:
        bind(t1, t2); return  $\theta$ 
    si t2 es variable libre:
        bind(t2, t1); return  $\theta$ 
    si t1 = f(a1..an) y t2 = f(b1..bn):
        para cada i: unify(ai, bi)
        return  $\theta$ 
    FALLA

```

Importante: cuando se unifica una variable con un término, **todas las ocurrencias** posteriores de esa variable usan el término asignado.

3.1.3 MGU (sustitución más general)

El algoritmo de Robinson produce la **MGU**: la sustitución más general que hace coincidir ambos términos. No cualquier sustitución, **la** más libre.

Formalmente: θ es MGU si para cualquier otra σ que unifique t_1 y t_2 , existe ρ tal que $\sigma = \theta \circ \rho$.

Práctica: esto importa porque las variables quedan lo más libres posible para el resto de la resolución — más flexibilidad para el resto del programa.

3.1.4 Occurs-check: el gotcha

Problema: ¿qué pasa con `?- X = f(X).` ?

- Sin occurs-check (SWI-Prolog por defecto): crea un **término cíclico** `X = f(f(f(...)))`, que Prolog representa internamente pero puede causar loops en algoritmos que iteran.
- Con occurs-check: la unificación **falla** — es la semántica correcta.

Por qué SWI lo desactiva: el occurs-check agrega costo lineal a cada unificación. En 99% de los programas no hace falta. Pero si estás trabajando con estructuras que pueden generar ciclos → actívalo:

```

?- set_prolog_flag(occurs_check, true).
?- unify_with_occurs_check(X, f(X)).    % también explícito

```

3.1.5 Los operadores `=`, `==`, `\=`, `\==`, `==..`

Operador	Semántica	Efecto en variables
<code>=/2</code>	Unifica	Las liga
<code>==/2</code>	Idénticos estructuralmente	No las liga; compara
<code>\=/2</code>	No unifican	—
<code>\==/2</code>	No idénticos	—
<code>=../2</code>	Descompone término a lista	Liga el resultado

Ejemplos críticos para el parcial:

```
?- X = 5.           X = 5.
?- X == 5.          false.      % X es libre, 5 es un número
?- X = 5, X == 5.   true.       % tras ligar, son idénticos
?- padre(juan, X) =.. L.
                    L = [padre, juan, X].
?- T =.. [f, a, b].
                    T = f(a, b).
```

3.2 Resolución SLD

SLD = **S**elective **L**inear resolution for **D**efinite clauses.

3.2.1 Algoritmo intuitivo

Dado un programa P y una consulta (goal) G :

1. Comenzar con `resolvente = [G]`.
2. Mientras resolvente no esté vacía:
 - a. Elegir el goal **más a la izquierda** (regla de selección Prolog).
 - b. Buscar en P una cláusula `H :- B1, ..., Bn.` cuya cabeza H unifique con el goal (MGU = θ).
 - c. Reemplazar el goal en la resolvente por $B_1, \dots, B_n$ (con θ aplicada).
 - d. Acumular sustituciones.
3. Si resolvente vacía → **éxito** con las sustituciones.
4. Si ningún goal unifica → **backtrack** al último punto de elección.

3.2.2 Árbol SLD

Cada paso ramifica si hay múltiples cláusulas que unifican. El conjunto de caminos forma un árbol:

- Raíz = consulta original
- Cada nodo = una resolvente
- Cada arco = aplicación de una cláusula
- Hojas verdes = resolvente vacía (soluciones)
- Hojas rojas = fallo (ningún goal unifica)

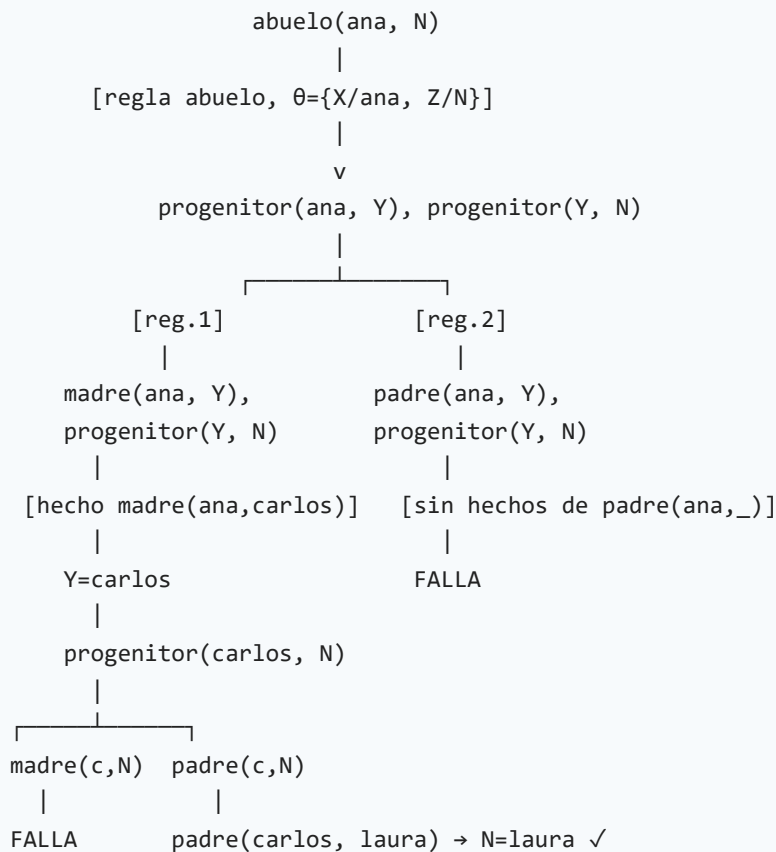
Prolog recorre este árbol en **DFS** (depth-first search), de izquierda a derecha.

Ejemplo completo:

Base:

```
madre(ana, carlos).
madre(ana, beatriz).
padre(carlos, laura).
progenitor(X,Y) :- madre(X,Y).
progenitor(X,Y) :- padre(X,Y).
abuelo(X,Z) :- progenitor(X,Y), progenitor(Y,Z).
```

Consulta: `?- abuelo(ana, N).`



La primera solución es `N = laura`. Si el usuario pide más (`;`), Prolog backtrackea y busca otra.

3.2.3 Por qué el orden importa

La regla de selección **leftmost** + la regla de escritura **top-down** tienen consecuencias:

Caso 1 — caso base primero:

```
ancestro(X,Y) :- progenitor(X,Y).           % base
ancestro(X,Y) :- progenitor(X,Z), ancestro(Z,Y). % recursivo
```

Funciona siempre. Encuentra respuestas cortas primero.

Caso 2 — recursivo primero:

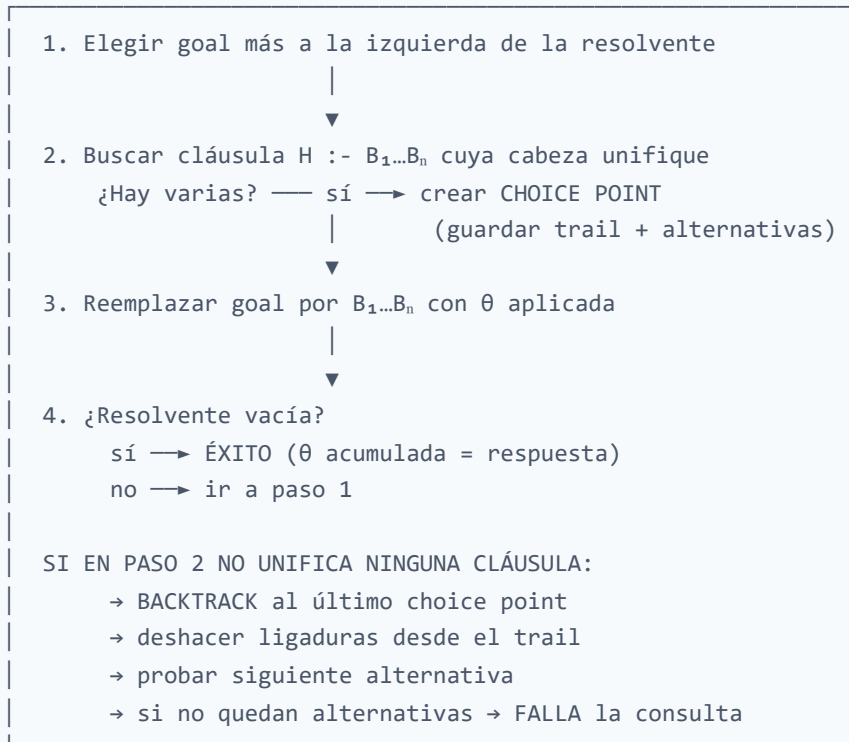
```
ancestro(X,Y) :- progenitor(X,Z), ancestro(Z,Y).
ancestro(X,Y) :- progenitor(X,Y).
```

Todavía es lógicamente correcto, pero Prolog puede entrar en **loop infinito** explorando recursiones arbitrarias antes de tocar el caso base.

Regla práctica: siempre caso base antes del recursivo.

3.3 Backtracking

 **Mapa mental del ciclo de backtracking** (tenelo a mano mientras estudiás la sección):



Regla mnemotécnica: “elegir, unificar, consumir — si falla, trail y choice point”.

3.3.1 El concepto

Cuando una rama del árbol SLD falla, Prolog retrocede al último **punto de elección** (*choice point*) y prueba la siguiente alternativa. Es **automático**.

Cuándo se crea un choice point: - Múltiples cláusulas unifican con un goal - Un predicado tiene múltiples hechos (p. ej. `color(rojo).` `color(verde).`)

Cómo revertir las ligaduras: Prolog mantiene un **trail** (registro de ligaduras). Al hacer backtrack, deshace las ligaduras desde el trail hasta el choice point.

3.3.2 El patrón `fail` -driven loop

Clásico pero algo anticuado:

```
?- color(X), write(X), nl, fail.
```

- `color(X)` encuentra `rojo` , `write` lo imprime, `fail` fuerza backtrack.
- `color(X)` ahora devuelve `verde` , etc.
- Cuando no quedan colores, todo falla → el `?-` responde `false` .

Alternativas modernas: `forall/2` , `findall/3` , `maplist/2` .

3.3.3 Eficiencia y explosión combinatoria

Si tenés 5 predicados independientes con 4 soluciones cada uno → $4^5 = 1024$ ramas.

Optimización natural: - Poner goals restrictivos **primero** (filtrar temprano). - Diseñar hechos con cláusulas discriminantes (el primer argumento es el más útil — SWI indexa automáticamente por ahí).

3.4 El corte (!)

3.4.1 Efecto

! es un goal que: - **Siempre tiene éxito** la primera vez que se ejecuta. - **Elimina** los choice points de la cláusula actual desde su inicio hasta la posición del **!** . - **Elimina** el choice point que permitiría probar otras cláusulas del mismo predicado.

En otras palabras: *"Ya decidí. No vuelvas atrás."*

3.4.2 Corte verde vs. rojo

Verde: no cambia la semántica del programa, sólo la eficiencia.

```
max(X, Y, X) :- X >= Y, !.  
max(_, Y, Y).
```

Si quitás el `!`, sigue siendo correcto — solo habrá un choice point inútil.

Rojo: cambia la semántica. Quitarlo rompe el programa.

```
abs(X, X) :- X >= 0, !.  
abs(X, Y) :- Y is -X.
```

Sin `!`, `?- abs(3, Z).` daría `Z=3 ; Z=-3` (erróneo).

Regla de oro: preferir corte verde. Si necesitás rojo, documentalo con comentario.

● Verde vs rojo — tabla de bolsillo

Criterio	Verde	Rojo
¿Cambia resultados si lo sacás?	No	Sí
¿Rompe semántica declarativa?	No	Sí
Intención	Optimización	Control de flujo
Recomendación	OK	Refactorizar a <code>-></code> si se puede
Test para distinguirlos	Correr consultas sin el corte: si dan mismos resultados → verde	Si cambian / aparecen soluciones extras → rojo

Contraejemplo ejecutable: corré este programa mentalmente **con** y **sin** `!`:

```
clasificar(X, positivo) :- X > 0, !.  
clasificar(X, negativo) :- X < 0, !.  
clasificar(_, cero).  
  
?- clasificar(5, C).
```

- **Con** `!` → `C = positivo.` (y punto: no explora más)
- **Sin** `!` → `C = positivo ; C = cero.` (dos respuestas — la segunda es **errónea**)

Los cortes aquí son **rojos** porque filtran resultados. Reescritura correcta sin corte rojo:

```
clasificar(X, positivo) :- X > 0.  
clasificar(X, negativo) :- X < 0.  
clasificar(X, cero)      :- X == 0.
```

Ahora las condiciones son **mutuamente excluyentes** — el programa es declarativo puro.

3.4.3 Alternativa moderna: `(Cond -> Then ; Else)`

```
abs(X, Y) :-  
  ( X >= 0  
  -> Y = X  
  ; Y is -X  
  ).
```

Ventajas: - **Local** (no afecta a otras cláusulas) - **Más legible** - **Sintáctica standard** (ISO)

3.5 Negación por falla

3.5.1 `\+` Goal

Significa: “Goal no es demostrable con el programa actual.”

No es “Goal es falso en el mundo”. Es la **Closed World Assumption (CWA)**: asumimos que todo lo no demostrado es falso.

3.5.2 Trampas

Trampa 1 — mundo abierto:

```
casado(ana).  
soltero(X) :- \+ casado(X).  
  
?- soltero(juan).    true.           % aunque podríamos no saberlo
```

Trampa 2 — variables libres:

```
?- \+ X = 1, X = 2.  
false.  
  
?- X = 2, \+ X = 1.  
true.
```

El orden cambia la respuesta porque `\+` depende del estado de las variables.

Regla: usar `\+` sólo con goals **ground** (sin variables libres).

3.5.3 `dif/2` como alternativa correcta

```
?- dif(X, Y), X = 1, Y = 2.    true.  
?- dif(X, Y), X = 1, Y = 1.    false.
```

`dif/2` se **pospone** hasta que las variables estén ligadas y entonces verifica. Es la negación correcta en CLP.

3.6 Aritmética

3.6.1 El momento incómodo

```
?- X = 2 + 3.
X = 2+3.      % el término, no 5
```

Prolog **no evalúa** expresiones automáticamente. Trata `2 + 3` como el término `+(2, 3)`.

3.6.2 `is/2` : evaluación forzada

```
?- X is 2 + 3.      X = 5.
```

Reglas: - `is/2` **evalúa** el lado derecho aritméticamente. - El lado derecho debe ser **ground** (sin variables libres). - No es reversible: `?- 5 is X + 3.` da error.

3.6.3 Operadores y comparadores

Categoría	Operadores	Evalúan
Aritméticos	<code>+ - * / // mod **</code>	Con <code>is/2</code>
Comparadores	<code>== \= < <= > >=</code>	Sí (ambos lados)
Unificación	<code>= \= == \==</code>	No

Trampas clásicas: - `<=` (no `<=`) - `==` vs. `==` vs. `=` (tres cosas distintas, mirá sección 3.1.5) - `2 + 3 == 5` es **false** (`+(2,3)` no es idéntico a `5`) - `2 + 3 == 5` es **true** (evaluación numérica)

3.6.4 Factorial correcto

```
factorial(0, 1).
factorial(N, F) :-
    N > 0,
    N1 is N - 1,
    factorial(N1, F1),
    F is N * F1.
```

Observa: - `N > 0` antes de la recursiva (terminación) - `N1 is N - 1` antes de la recursiva (ground en la llamada) - `F is N * F1` después (cuando `F1` ya está ligado)

Anti-patrón: `factorial(N, N * factorial(N-1))`. — Prolog guarda el árbol de términos pero **nunca evalúa**.

3.7 Listas

3.7.1 Representación

Una lista es una **cons list**: cada elemento es una estructura con funtor `./2` (punto).

- `[]` — lista vacía
- `[a, b, c] = '.'(a, '.'(b, '.'(c, []))) = [a | [b | [c | []]]]`
- `[H | T]` — patrón: cabeza, cola

3.7.2 Primitivas fundamentales

member/2 :

```
member(X, [X|_]).  
member(X, [_|T]) :- member(X, T).
```

Multi-modo: pregunta y genera.

append/3 :

```
append([], L, L).  
append([H|T], L, [H|R]) :- append(T, L, R).
```

Reversible: concatena, divide, enumera prefijos.

length/2 :

```
length([], 0).  
length(_|T, N) :- length(T, N1), N is N1 + 1.
```

3.7.3 El poder de `append/3`

Con una sola definición, podés:

```
?- append([1,2], [3,4], R).           % concatenar  
R = [1,2,3,4].  
  
?- append(A, B, [1,2,3]).           % dividir  
A = [], B = [1,2,3] ; A = [1], B = [2,3] ; ...  
  
?- append(_, [X|_], [a,b,c,d]).     % enumerar elementos  
X = a ; X = b ; X = c ; X = d.  
  
?- append(_, [X, Y], [a,b,c,d]).    % últimos 2  
X = c, Y = d.
```

Clave pedagógica: `append/3` es un **ejemplo maestro** de cómo la lógica es más general que la función. Relaciona tres listas — podés fijar cualquier combinación.

3.8 Recursión con acumulador

 **Analogía de la mochila** (leé esto antes de cualquier código):

Imaginá que caminás un sendero recogiendo piedras. Tenés dos formas de contar el peso total:

Recursión ingenua — “*al final pregunto cuánto pesa todo*”: Llegás al final del sendero con las manos vacías y ahí empezás a sumar pesos de memoria hacia atrás. Si el sendero es larguísimo (100.000 piedras), no te alcanza la memoria.

Recursión con acumulador — “*llevo una mochila y voy cargando*”: al empezar la mochila pesa 0. En cada piedra: sumás su peso a la mochila y seguís. Cuando llegás al final, la mochila ya tiene el resultado — no necesitás recordar nada del camino.

Regla mental para escribirlo: - El **wrapper** inicializa la mochila: `suma(L, S) :- suma(L, 0, S).` - El **caso base** devuelve la mochila: `suma([], Acc, Acc).` - El **caso recursivo** carga la mochila *antes* de llamar: `Acc1 is Acc + H, suma(T, Acc1, S).`

Pregunta detectora (hacete esta pregunta mientras escribís): > “¿El cálculo está *ANTES* o *DESPUÉS* de la llamada recursiva?”

- **Antes** → tail-recursive (LCO se aplica, sin stack overflow) 
- **Después** → ingenuo (va a crecer el stack) 

3.8.1 El problema del stack

Recursión ingenua:

```
suma([], 0).  
suma([H|T], S) :- suma(T, ST), S is H + ST.
```

Con una lista de 100.000 elementos → **stack overflow**, porque cada llamada recursiva tiene que esperar el resultado para sumar.

3.8.2 La versión con acumulador

```
suma(L, S) :- suma(L, 0, S).  
  
suma([], Acc, Acc).  
suma([H|T], Acc, S) :-  
    Acc1 is Acc + H,  
    suma(T, Acc1, S).
```

La clave: la llamada recursiva es la **última operación**.

3.8.3 Last-Call Optimization (LCO)

Cuando la última llamada es recursiva y no hay choice points pendientes, el motor Prolog **reutiliza el stack frame**. Resultado: recursión en **memoria constante**, equivalente a un loop imperativo.

Todos los Prologs modernos la implementan. Por eso "recursión en Prolog" NO es más lenta que un `while`.

3.8.4 Reverse con acumulador

```
reverse(L, R) :- rev(L, [], R).  
rev([], Acc, Acc).  
rev([H|T], Acc, R) :- rev(T, [H|Acc], R).
```

Trazado mental para `[1,2,3]`:

```
rev([1,2,3], [], R)  
rev([2,3], [1], R)  
rev([3], [2,1], R)  
rev([], [3,2,1], R)  
R = [3,2,1].
```

O(n), memoria constante.

3.9 Meta-predicados

3.9.1 findall/3

```
findall(+Template, +Goal, -List).
```

Colecta **todas** las instancias de `Template` para las que `Goal` es verdadero. Si no hay soluciones, `List = []`.

```
?- findall(N, edad(N, 22), L).  
L = [ana, carla].
```

3.9.2 bagof/3 y setof/3

`bagof/3`: como `findall` pero **falla** si no hay soluciones + **agrupa** por variables libres.

`setof/3`: como `bagof` pero **ordenado** y **sin duplicados**.

```
?- setof(N, E^edad(N, E), L).
L = [ana, beto, carla].
```

El operador `^` cuantifica existencialmente — dice “no me importa el valor de E”.

3.9.3 Tabla comparativa

Aspecto	findall/3	bagof/3	setof/3
Sin soluciones	<code>[]</code>	falla	falla
Duplicados	sí	sí	no
Orden	preserva	preserva	ordenado
Agrupar por vars libres	no	sí	sí

3.10 Aplicaciones canónicas

3.10.1 Generate and test

```
color(rojo). color(verde). color(azul).

mapa(N, S, E, O) :-
    color(N), color(S), color(E), color(O),
    N \= S, N \= E, S \= O, E \= O.
```

Genera todas las combinaciones, filtra las válidas. Combinado con CLP(FD) evita combinatoria explosiva.

3.10.2 Base deductiva con consultas múltiples

```
vuelo(ush, bue, 2200).
vuelo(bue, mvd, 150).

ruta(A, B, [A,B], T) :- vuelo(A, B, T).
ruta(A, B, [A|R], T) :-
    vuelo(A, C, T1), ruta(C, B, R, T2),
    T is T1 + T2.
```

Una base, múltiples preguntas: - ¿Hay vuelo directo? `?- vuelo(ush, bue, _)`. - ¿Cuál es la duración total mínima? - ¿Cuántas escalas? - ¿Hay alguna con escala en MVD?

Esto es lo que **ningún otro paradigma** te da: reversibilidad total.

3.10.3 CLP(FD) para puzzles

```
:- use_module(library(clpfd)).

n_reinas(N, Qs) :-
    length(Qs, N),
    Qs ins 1..N,
    all_distinct(Qs),
    diagonales_distintas(Qs),
    label(Qs).
```

CLP(FD) hace **búsqueda con restricciones** — evita la fuerza bruta. Es la técnica moderna para puzzles grandes.

4. Ejemplos trabajados paso a paso

Ejemplo 1 — Unificar estructuras

Unificar `par(X, punto(Y, 3))` con `par(5, punto(X, Z))`:

```
Paso 1: par(X, punto(Y,3)) ≡ par(5, punto(X,Z))
        Funtores iguales (par/2), unificar argumentos
Paso 2: X ≡ 5                                → bind {X/5}
Paso 3: punto(Y,3) ≡ punto(X,Z)
        Pero X ya vale 5. El segundo término se vuelve punto(5,Z).
        punto(Y,3) ≡ punto(5,Z)
        Funtores iguales (punto/2), unificar argumentos
Paso 4: Y ≡ 5                                → bind {Y/5}
Paso 5: 3 ≡ Z                                → bind {Z/3}
Resultado: {X/5, Y/5, Z/3}.                  ¡Éxito!
```

Ejemplo 2 — Trazar un backtracking

Base:

```
bebida(agua).
bebida(vino).
comida(pasta).
comida(ensalada).

almuerzo(B, C) :- bebida(B), comida(C), B \= C.
```

Consulta: `?- almuerzo(X, Y).`

Choice 1: B=agua, C=pasta	→ agua \= pasta → ✓ → X=agua, Y=pasta
[usuario pide más]	
Choice 2: B=agua, C=ensalada	→ ✓ → X=agua, Y=ensalada
Choice 3: B=vino, C=pasta	→ ✓ → X=vino, Y=pasta
Choice 4: B=vino, C=ensalada	→ ✓ → X=vino, Y=ensalada
No más bebidas ni comidas	→ false.

4 soluciones. El orden está determinado por DFS + top-down.

Ejemplo 3 — Escribir `member/2` desde cero

Análisis: - ¿Qué significa “X es miembro de L”? - O bien `X` es la cabeza de `L` - O bien `X` es miembro de la cola

Codificación:

```
member(X, [X|_]).           % caso 1: X es la cabeza
member(X, [_|T]) :- member(X, T). % caso 2: X está en la cola
```

Prueba:

```
?- member(b, [a,b,c]).
Intenta cláusula 1: member(b, [a|_]) - requiere b=a → falla
Backtrack a cláusula 2: member(b, [_|T]) con T=[b,c]
Llamada recursiva: member(b, [b,c])
Intenta cláusula 1: member(b, [b|_]) → ÉXITO
true.
```

Ejemplo 4 — `reverse` con acumulador

Queremos invertir `[1, 2, 3]` a `[3, 2, 1]`.

Idea del acumulador: ir “rotando” la lista hacia el acumulador.

```
reverse(L, R) :- rev(L, [], R).

rev([], Acc, Acc).
rev([H|T], Acc, R) :- rev(T, [H|Acc], R).
```

Trazado:

```

rev([1,2,3], [], R)
  H=1, T=[2,3], Acc=[]
  llama rev([2,3], [1], R)
    H=2, T=[3], Acc=[1]
    llama rev([3], [2,1], R)
      H=3, T=[], Acc=[2,1]
      llama rev([], [3,2,1], R)
        caso base: R = [3,2,1]

```

Observar: cada cabeza va al frente del acumulador → al llegar al final, el acumulador tiene la lista invertida.

Ejemplo 5 — `findall` para promediar

```

edad(ana, 22). edad(beto, 30). edad(carla, 22).

promedio_edad(P) :-
  findall(E, edad(_, E), L),
  length(L, N),
  sum_list(L, S),
  P is S / N.

```

Trazado: 1. `findall(E, edad(_, E), L)` → `L = [22, 30, 22]` 2. `length(L, 3)` → `N = 3` 3. `sum_list(L, 74)` → `S = 74` 4. `P is 74 / 3` → `P ≈ 24.67`

Ejemplo 6 — Resolver coloreo de 4 regiones

Base:

```

color(rojo). color(verde). color(azul). color(amarillo).
distinto(A, B) :- A \= B.

mapa(N, S, E, O) :-
  color(N), color(S), color(E), color(O),
  distinto(N, S), distinto(N, E),
  distinto(S, O), distinto(E, O).

```

Consulta:

```

?- mapa(N, S, E, O).
N = rojo, S = verde, E = verde, O = rojo ;
N = rojo, S = verde, E = verde, O = amarillo ;
...

```

Prolog enumera todas las combinaciones válidas gracias al backtracking.

5. Puntos clave (lo que tenés que saber sí o sí)

1. **La unificación se define por el algoritmo de Robinson** (4 casos). Memorizalos.
 2. `=`, `==`, `==:`, `is/2` **son cuatro cosas distintas**. No las confundas:
 - `=` : unifica (liga variables)
 - `==` : compara idéntico sin ligar
 - `==:` : evalúa ambos lados y compara numéricamente
 - `is/2` : evalúa el lado derecho y unifica con el izquierdo
 3. **El orden de las cláusulas importa** para terminación y eficiencia: **caso base antes del recursivo**.
 4. **Backtracking es automático y destructivo**: deshace ligaduras al retroceder.
 5. **Corte verde vs. rojo**: verde no cambia semántica; rojo sí. Preferir `(-> ;)`.
 6. `\+` **no es negación lógica** — es falla demostrativa (CWA). Solo con goals ground.
 7. **Aritmética requiere `is/2` explícito**. Prolog nunca evalúa por defecto.
 8. `append/3` **es reversible** — entiéndelo y tendrás Prolog en el bolsillo.
 9. **Recursión con acumulador** → **LCO** → **memoria constante**.
 10. `findall/bagof/setof` : usar `findall` salvo agrupación o unicidad.
-

6. Autoevaluación (20 preguntas)

Respondé sin mirar. Las respuestas están al final.

1. ¿Qué devuelve `?- X = 5, X == 5.` ?
2. ¿Y `?- X == 5.` ?
3. Escribir la MGU de `f(X, a, Y)` con `f(b, Z, c)`.
4. ¿`?- X = f(X).` en SWI-Prolog por defecto: éxito o falla?
5. ¿Qué imprime `?- color(X), write(X), nl, fail.` si hay 3 colores?
6. ¿Qué es la CWA?
7. ¿Por qué `?- \+ X = 1, X = 2.` falla?
8. ¿Qué diferencia hay entre `?- 2+3 = 5.` y `?- 2+3 == 5.` ?
9. Escribir `member/2` desde cero.

10. Escribir `append/3` desde cero.
11. ¿Qué es LCO?
12. Convertir `suma([], 0). suma([H|T], S) :- suma(T, ST), S is H + ST.` en versión con acumulador.
13. ¿Qué diferencia hay entre `findall/3` y `bagof/3` cuando no hay soluciones?
14. ¿Qué hace `=../2`?
15. Dado `?- setof(E, N^edad(N, E), L).`, ¿qué significa el `^`?
16. ¿Un corte verde cambia las respuestas del programa?
17. ¿Cuál es la alternativa moderna al corte para codificar if-then-else?
18. Escribir una regla Prolog que diga "X es tío de Y si X es hermano de un progenitor de Y".
19. ¿Qué predicado usarías para forzar aritmética reversible ($X + Y = 10$ con X, Y variables)?
20. Nombrar 3 implementaciones modernas de Prolog.

Respuestas

1. `true`.
2. `false` (X es variable libre, 5 es número; no son idénticos).
3. `{X/b, Z/a, Y/c}`.
4. Éxito (sin occurs-check, crea término cíclico).
5. `rojo\nverde\nazu1\n` y luego `false`.
6. Closed World Assumption: lo no demostrado se asume falso.
7. Porque al ejecutar `\+ X = 1` con X libre, Prolog demuestra `X = 1` (con X ligado temporalmente) $\rightarrow$ `\+` falla.
8. `=` intenta unificar `+(2,3)` con `5` $\rightarrow$ falla. `==` evalúa ambos $\rightarrow$ `5 = 5` $\rightarrow$ true.
9. Ver sección 3.7.2.
10. Ver sección 3.7.2.
11. Last-Call Optimization: el motor reutiliza el stack frame si la última llamada es recursiva y no hay choice points.
12. Ver sección 3.8.2.
13. `findall/3` devuelve `[]`; `bagof/3` falla.
14. Descompone un término en lista `[functor | args]`.
15. Cuantificación existencial: "para algún N cualquiera".
16. No.
17. `(Condición -> Then ; Else)`.
18. `tio(X, Y) :- hermano(X, P), progenitor(P, Y).`
19. `library(clpfd)` con `#=`.
20. SWI-Prolog, Stryer Prolog, Trealla Prolog, Tau Prolog, Ciao.

Calificación: - 18–20: excelente, listo para el TP. - 14–17: bien, revisá 2–3 puntos flojos. - 10–13: releer secciones 3.1, 3.4, 3.7, 3.8. - <10: re-estudiar desde la sección 3.

7. Glosario

Término	Definición
Átomo	Constante (minúscula o entre comillas)
Backtracking	Retroceso automático al último choice point
Base de conocimiento	Conjunto de hechos y reglas
CLP(FD)	Constraint Logic Programming over Finite Domains
Cláusula de Horn	Implicación con una sola cabeza
Choice point	Punto del árbol donde Prolog guarda alternativas
CWA	Closed World Assumption
Datalog	Subconjunto restringido y terminante de Prolog
DCG	Definite Clause Grammar (parsers en Prolog)
DFS	Depth-first search (búsqueda en profundidad)
Ground	Término sin variables libres
LCO	Last-Call Optimization
MGU	Most General Unifier
Occurs-check	Verificación de ciclos en unificación
Predicado	Relación entre argumentos (p. ej. <code>padre/2</code>)
Resolvente	Conjunción de goals pendientes
SLD	Selective Linear resolution for Definite clauses
Sustitución	Mapa de variable → término
Trail	Registro de ligaduras para backtracking
Unificación	Matching con pega — encontrar sustitución común

8. Referencias

Obligatorias

1. **Sebesta, R.** (2019). *Concepts of Programming Languages*, 12th ed. Pearson. **Cap. 16** (pp. 703–784).
2. **Gabbrielli, M. & Martini, S.** (2023). *Programming Languages: Principles and Paradigms*, 2nd ed. Springer. **Cap. Logic Programming** (pp. 351–423).
3. **Louden, K. & Lambert, K.** (2011). *Programming Languages: Principles and Practices*, 3rd ed. Cengage. **Cap. 4 Logic Programming**.

Recomendadas

4. **Sterling, L. & Shapiro, E.** (1994). *The Art of Prolog*, 2e. MIT Press. Caps. 3, 6, 8.
5. **Bratko, I.** (2012). *Prolog Programming for Artificial Intelligence*, 4e. Pearson.
6. **Spivey, M.** (2005). *An Introduction to Logic Programming through Prolog*. Caps. 4–6.

Online

- **SWI-Prolog Docs:** <https://www.swi-prolog.org/pldoc/>
- **SWISH (playground):** <https://swish.swi-prolog.org/>
- **The Power of Prolog** (Markus Triska): <https://www.metalevel.at/prolog> (mejor recurso online 2020+)

Papers 2024–2026

- **DeepProbLog 2.0** (Manhaeve et al., 2024) — razonamiento probabilístico + redes neuronales.
- **AlphaProof** (DeepMind, 2024) — resolución formal con LLM + motor lógico.
- **Tau Prolog 2026** — integración con Node.js / navegador.

9. Siguientes pasos

Una vez cerrado este tema: 1. Completá el **TP** (2 semanas, ver `tp.md`). 2. Guardá esta guía — va a estar en el parcial. 3. Para el parcial: practicá **a mano** 5 trazados SLD. 4. Próxima clase: **Paradigma OO con TypeScript** (Tema 08). Preparate pensando: si Prolog es “relaciones”, OO es “mensajes y estado”.

Trazabilidad: [diseno.md](#) · [minuta.md](#) · [filminas.md](#) (158 slides) Fuentes: Sebesta, Gabbrielli & Martini, Louden, Sterling & Shapiro, The Power of Prolog